Studio Nx

P1-12 문의 API — 함수 설명 합본 (백엔드 50~55 · JS 28)

2026-10-12

목차

1. ai-notes/backend(fastAPI_Python)/P1-12 - 문의 API/50_inquiry_schema_request.md
2. ai-notes/backend(fastAPI_Python)/P1-12 - 문의 API/51_inquiry_service_rate_limit.md
3. ai-notes/backend(fastAPI_Python)/P1-12 - 문의 API/52_inquiry_service_create_inquiry.md
4. ai-notes/backend(fastAPI_Python)/P1-12 - 문의 API/53_inquiries_route_create.md
5. ai-notes/backend(fastAPI_Python)/P1-12 - 문의 API/54_retention_purge_old_inquiries.md
6. ai-notes/backend(fastAPI_Python)/P1-12 - 문의 API/55_retention_main.md
7. ai-notes/javascript/P1-12 - 문의 폼 JS/28_home_initContactForm.md

Backend(FastAPI/Python) 50 — `InquiryCreateRequest` (문의 요청 스키마)

항목	내용
파일	`studionx/backend/app/schemas/inquiry.py`
작업	P1-12 문의 API: 요청 본문 검사 (PRD 01 FR-112)
작성	Claude (AI), 2026-10-08, 사용자 검토
이전 · 다음	49 `purge_withdrawn_accounts()` · 51 `check_rate_limit()`

1. 하는 일

홈 화면의 **Contact** 폼이 보내는 JSON `{ email, title, content, privacy_agreed }`의 **모양**을 검사합니다. 값이 틀리면 서비스까지 가지 않고 422로 응답합니다.

보낸 값	결과
`" you@example.com "`	통과하고, 앞뒤 공백을 지운 `"you@example.com"`이 됩니다
이메일이 `abc`, `kim@naver`, `a@b@c.com`, 빈 문자열	422이고, `details.email`에 오류가 담깁니다
제목 `" Bug report "`	통과하고, `"Bug report"`가 됩니다
제목이나 내용이 빈 문자열, 공백만, 줄바꿈만	422이고, 해당 칸에 오류가 담깁니다
내용 `"\n line 1\nline 2 \n"`	통과하고 `"line 1\nline 2"`가 됩니다. **가운데 줄바꿈은 그대로 남습니다**
제목 201자, 내용 2001자	422입니다 (앞뒤 공백은 길이에 세지 않습니다)
`privacy_agreed`가 없음	422입니다. `false`는 여기서 통과하고, 서비스(52)가 422로 막습니다

2. 언제 쓰나

쓰는 곳	
`POST /api/v1/inquiries` (53)	라우터 매개변수 `data: InquiryCreateRequest`로 씁니다. 값이 틀리면 함수가 호출되지도 않고 422로 응답합니다 (P1-03 03번 처리기)
화면	home.html의 Contact 폼(`initContactForm`, 이 작업이 끝나면 연결)입니다. 화면도 `type="email"`, `required`, `maxlength`로 먼저 막고, FIX-01 19번이 브라우저 말풍선 대신 알림창으로 보여 줍니다

3. 코드

import (파일 맨 위)

```
from datetime import datetime
from typing import Annotated

from pydantic import BaseModel, EmailStr, Field, StringConstraints
```

`Annotated`, `EmailStr`, `StringConstraints`를 이번에 추가했습니다.

상수와 타입

```
INQUIRY_TITLE_MAX = 200  # 화면 input maxlength 와 같음
INQUIRY_CONTENT_MAX = 2000  # 화면 textarea maxlength 와 같음

# 제목 · 내용 — 앞뒤 공백을 먼저 지우고 길이를 잼 → 공백만 친 칸은 422
# (닉네임 Nickname 과 같은 방법)
# 내용 가운데의 줄바꿈은 그대로 둠
InquiryTitle = Annotated[
    str, StringConstraints(strip_whitespace=True, min_length=1, max_length=INQUIRY_TITLE_MAX)
]
InquiryContent = Annotated[
    str, StringConstraints(strip_whitespace=True, min_length=1, max_length=INQUIRY_CONTENT_MAX)
]
```

스키마

```
class InquiryCreateRequest(BaseModel):
    email: EmailStr = Field(max_length=255)  # 형식 검사 · 앞뒤 공백 제거 (email-validator)
    title: InquiryTitle
    content: InquiryContent
    privacy_agreed: bool
```

문법 풀이

import 문법	뜻
`from typing import Annotated`	타입에 **검사 규칙을 붙인** 새 타입 이름을 만들 때 씁니다
`from pydantic import EmailStr, StringConstraints`	앞은 이메일 형식 타입(P1-04와 같음, `email-validator` 패키지)이고, 뒤는 글자 규칙 묶음입니다

식	뜻
`Annotated[str, StringConstraints(…)]`	"문자열이지만 이 규칙을 지켜야 한다"는 뜻입니다. 변수처럼 이름(`InquiryTitle`)을 붙여 여러 곳에서 씁니다
`strip_whitespace=True`	앞뒤의 공백, 줄바꿈, 탭을 **먼저** 지우고, 그다음에 `min_length`와 `max_length`를 검사합니다
`EmailStr = Field(max_length=255)`	이메일 형식과 길이(DB 칸 255자)를 함께 검사합니다
`privacy_agreed: bool`	반드시 있어야 하는 칸입니다 (기본값 없음). 값이 `true`인지는 서비스가 확인합니다

4. 흐름

요청 JSON이 들어오면 pydantic이 칸마다 이메일 형식, 공백 제거, 길이, 동의 칸 존재 여부를 검사합니다. 모두 통과하면 정리된 값이 담긴 객체가 라우터로 전달되고, 하나라도 틀리면 422와 함께 `details`에 틀린 칸이 담깁니다.

5. 왜 이렇게

선택	이유
이메일 형식 검사에 `EmailStr` 사용	템플릿이 "P1-04 SignupRequest와 같은 방법"을 요구했습니다. 답장을 보낼 주소이므로 형식이 틀리면 받을 의미가 없습니다
제목과 내용을 **공백을 지운 뒤** 검사	템플릿대로(`min_length=1`) 두면 공백 한 칸짜리 문의가 저장됩니다. 닉네임(P1-08 37)에 있던 것과 같은 빈틈입니다
앞뒤 공백만 지움	내용 안의 줄바꿈과 문단은 문의 내용의 일부라서 그대로 둡니다 (시험으로 확인했습니다)
길이 상수를 화면과 같게 맞춤 (`INQUIRY_TITLE_MAX` 신설)	화면의 `maxlength`(200, 2000)와 서버 기준이 어긋나지 않게 했습니다. 제목 길이는 템플릿에 숫자로만 적혀 있었습니다
이메일을 소문자로 바꾸지 않음	문의 이메일은 중복 확인이나 로그인에 쓰지 않으므로, 사람이 적은 그대로 보관합니다 (반복 전송 제한(51)이 비교할 때만 소문자로 바꿉니다)
`privacy_agreed=false`는 여기서 막지 않음	템플릿의 설계를 따랐습니다. 서비스(52)가 "Please agree to the privacy policy."와 `details.privacy_agreed`로 알려 줍니다 (가입과 같은 방식)

함께 생기는 변화: 52번 `create_inquiry` 템플릿에 있던 `data.title.strip()`은 이제 하지 않아도 됩니다. 이미 공백이 지워진 값이 오기 때문입니다.

6. 시험 (`tests/test_inquiry_schemas.py` 새 파일, 16개)

시험	확인한 것
정상	네 칸이 그대로 들어옵니다
공백 제거	제목의 앞뒤 공백을 지우고, 내용도 앞뒤만 지우며 가운데 줄바꿈은 유지하고, 이메일의 앞뒤 공백도 지웁니다
이메일 4가지	`@` 없음, 점 없음, `@` 두 개, 빈 문자열은 모두 `email` 오류입니다
빈 제목과 내용 6가지	빈 문자열, 공백만, 줄바꿈만인 경우를 두 칸에 각각 넣으면 해당 칸 오류입니다
길이	200자와 2000자는 통과하고, 201자와 2001자는 오류이며, 앞뒤 공백을 붙인 2000자는 통과합니다
동의 칸 없음	오류입니다

전체 시험 368개가 통과했고, `ruff` 검사도 통과했습니다.

Backend(FastAPI/Python) 51 — `check_rate_limit()` · `record_inquiry_send()`

항목	내용
파일	`studionx/backend/app/services/inquiry_service.py`
작업	P1-12 문의 API: 반복 전송 제한 (PRD 01 FR-113)
작성	Claude (AI), 2026-10-08, 사용자 검토
이전 · 다음	50 `InquiryCreateRequest` · 52 `create_inquiry()`

1. 하는 일

같은 사람이 짧은 시간 안에 문의를 계속 보내지 못하게 막습니다. **같은 이메일이나 같은 IP로 10분 안에 2번** 저장되었다면, 3번째는 429로 응답합니다.

상황 (같은 이메일, 같은 IP)	결과
1번째, 2번째	통과합니다
3번째 (10분 안)	429 `TOO_MANY_REQUESTS` "Too many messages. Please try again later."이고, `details.retry_after`에 몇 초 뒤에 보낼 수 있는지 담깁니다
IP는 다르지만 이메일이 같음 (`A@Example.com` 도 같은 이메일로 봅니다)	막힙니다
이메일은 다르지만 IP가 같음	막힙니다
IP도 이메일도 다름	통과합니다
저장에 **실패한** 시도와 막힌 시도	**세지 않습니다**
첫 번째 저장으로부터 10분이 지남	한 자리가 비어서 다시 1번 보낼 수 있습니다

2. 언제 쓰나

52번 `create_inquiry` 안에서 다음 순서로 부릅니다.

순서	함수
① 저장하기 전	`check_rate_limit(ip, email)`: 이미 2번 저장되었다면 여기서 429로 응답합니다 (저장하지 않습니다)
② 저장(commit)에 성공한 뒤	`record_inquiry_send(ip, email)`: 이번 전송을 셉니다

사용자 결정 (2026-10-08)

정한 것	결정	템플릿
횟수와 시간	**10분에 2번**	10분에 3번
저장에 실패한 시도도 세는가	**세지 않음**	(정하기)

템플릿의 `check_rate_limit`는 함수 하나가 "검사하고, 통과하면 바로 센다"는 구조였습니다. 그렇게 하면 검사를 통과한 뒤 **저장에 실패해도** 이미 세어진 상태가 됩니다. 결정대로 하려면 세는 일을 저장 **뒤로** 옮겨야 해서, 함수를 둘로 나눴습니다.

기록은 메모리에 두는 방식이며, 이것은 2026-10-03 결정을 그대로 따른 것입니다. 그래서 서버를 다시 켜면 기록이 비고, 서버가 여러 대라면 각자 따로 셉니다 (Phase 1은 서버 한 대).

3. 코드

import (파일 맨 위)

```
import threading
import time
from collections import deque

from sqlalchemy.orm import Session

from app.core.errors import AppError, ErrorCode
from app.models import Inquiry, User
from app.schemas.inquiry import InquiryCreateRequest
```

`threading`, `time`, `AppError`, `ErrorCode`를 이번에 추가했습니다.

상수와 기록 장소

```
INQUIRY_LIMIT_COUNT = 2  # 이 횟수까지 (2026-10-08 사용자 결정: 10분에 2번 — 템플릿은 3)
INQUIRY_LIMIT_SECONDS = 10 * 60  # 이 시간(초) 안에

_recent_sends: dict[str, deque[float]] = {}
_lock = threading.Lock()  # FastAPI 동기 함수는 여러 스레드에서 돎 → 기록을 함께 고치지 않게
_clock = time.monotonic  # 시스템 시계를 바꿔도 영향 없는 시각 (시험에서 바꿔 끼움)


def _keys(ip: str | None, email: str) -> list[str]:
    """제한을 셀 열쇠 — 이메일(소문자) 하나 + IP 가 있으면 IP 하나."""
    keys = [f"email:{email.strip().lower()}"]
    if ip:
        keys.append(f"ip:{ip}")
    return keys
```

함수

```
def check_rate_limit(ip: str | None, email: str) -> None:
    now = _clock()
    with _lock:
        for key in _keys(ip, email):
            q = _recent_sends.get(key)
            if q is None:
                continue
            while q and now - q[0] > INQUIRY_LIMIT_SECONDS:
                q.popleft()
            if not q:
                del _recent_sends[key]
                continue
            if len(q) >= INQUIRY_LIMIT_COUNT:
                raise AppError(
                    ErrorCode.TOO_MANY_REQUESTS,
                    429,
```

```
                    "Too many messages. Please try again later.",
                    details={"retry_after": int(INQUIRY_LIMIT_SECONDS - (now - q[0])) + 1},
                )


def record_inquiry_send(ip: str | None, email: str) -> None:
    now = _clock()
    with _lock:
        for key in _keys(ip, email):
            _recent_sends.setdefault(key, deque()).append(now)
```

문법 풀이

import 문법	뜻
`import threading`	여러 스레드가 같은 자료를 동시에 고치지 못하게 하는 잠금(`Lock`)을 씁니다
`import time`	`time.monotonic()`은 컴퓨터가 켜진 뒤 흐른 초입니다. 시계를 바꿔도 뒤로 가지 않습니다
`from collections import deque`	양쪽 끝에서 넣고 빼기가 빠른 목록입니다. 오래된 기록은 앞에서 빼고(`popleft`), 새 기록은 뒤에 넣습니다(`append`)

식	뜻
`dict[str, deque[float]]`	열쇠(`"email:a@b.com"`)마다 저장 시각 목록을 둡니다
`with _lock:`	이 블록은 한 번에 한 스레드만 실행합니다. 블록이 끝나면 잠금이 저절로 풀립니다
`_clock = time.monotonic`	함수를 **변수에 담아 둡니다**. 시험에서 손으로 움직이는 시계로 바꿔 끼우기 위해서입니다
`while q and now - q[0] > …: q.popleft()`	가장 오래된 기록부터 10분이 지난 것을 버립니다
`del _recent_sends[key]`	빈 목록은 열쇠째 지웁니다. 메모리가 계속 늘어나지 않게 하기 위해서입니다
`_recent_sends.setdefault(key, deque())`	열쇠가 없으면 빈 목록을 만들어 넣고, 있으면 그 목록을 돌려줍니다
`int(…) + 1`	남은 초를 올림합니다. 0초 남았다고 알려 주지 않기 위해서입니다

4. 흐름

검사: 지금 시각을 구하고 잠금을 겁니다. 열쇠(이메일, IP)마다 10분이 지난 기록을 버리고, 목록이 비면 지웁니다. 기록이 2개 이상이면 429로 응답합니다.

세기 (저장에 성공한 뒤): 지금 시각을 구하고 잠금을 건 다음, 열쇠마다 시각을 하나씩 추가합니다.

5. 왜 이렇게

선택	이유
검사와 세기를 둘로 나눔	"저장에 실패한 시도는 세지 않는다"는 사용자 결정에 따라, 세는 시점을 저장 뒤로 옮겼습니다
이메일 **또는** IP로 제한	이메일만 보면 주소만 바꿔서 계속 보낼 수 있고, IP만 보면 같은 사람이 와이파이를 바꿔 보낼 수 있습니다. 그래서 둘 다 봅니다
이메일은 소문자로 바꾸고 공백을 지워서 비교	`A@Example.com`과 `a@example.com`은 같은 사람이기 때문입니다

선택	이유
IP가 없으면 이메일로만 셈	시험 도구처럼 IP를 알 수 없는 요청을 위한 처리입니다. `ip:None`이라는 열쇠를 만들지 않아서, 모든 요청이 한 열쇠로 묶이는 일을 막습니다
`time.monotonic` 사용	서버 시계가 바뀌어도(시간 동기화 등) 10분 계산이 틀어지지 않습니다
잠금 사용	FastAPI의 동기 라우터는 여러 스레드에서 실행됩니다. 동시에 같은 목록을 고치면 기록을 잃을 수 있습니다 (시험: 스레드 100개)

알아 둘 점 (동시에 두 번 보내는 경우): 같은 사람이 **정확히 같은 순간에** 두 번 보내면, 둘 다 검사를 통과한 뒤 저장되어 한도보다 1번 더 저장될 수 있습니다. 검사부터 저장까지를 한 잠금으로 묶으면 막을 수 있지만, 그동안 다른 사람의 문의까지 기다려야 합니다. 문의 폼은 보내는 동안 버튼을 잠그기 때문에(화면) 실제로 일어날 일은 드뭅니다.

6. 시험 (`tests/test_inquiry_service.py` 새 파일, 12개, 시계를 손으로 움직이며 시험)

시험	확인한 것
상수	2번, 600초입니다 (사용자 결정)
3번째	429와 문구, `retry_after` 601초가 나옵니다
이메일이 같고 IP가 다름(대소문자와 공백도 다름), IP가 같고 이메일이 다름	둘 다 막힙니다
다른 사람	통과합니다
IP 없음	이메일로만 세고, `ip:None` 열쇠는 생기지 않습니다
검사만 5번 한 뒤(저장 실패) 실제 저장 2번	2번 모두 저장되고 3번째는 막힙니다. **실패한 시도는 세지 않습니다**
막힌 뒤 10분이 지남	다시 보낼 수 있습니다 (막힌 시도도 세지 않습니다)
`retry_after`	첫 기록을 기준으로 줄어듭니다
10분 범위가 밀림	첫 기록이 10분을 넘기면 1자리가 남습니다
오래된 기록	지워져서 기록 장소가 비게 됩니다
스레드 100개가 동시에 셈	기록을 하나도 잃지 않습니다

전체 시험 380개가 통과했고, `ruff` 검사도 통과했습니다.

Backend(FastAPI/Python) 52 — `create_inquiry()`

항목	내용
파일	`studionx/backend/app/services/inquiry_service.py`
작업	P1-12 문의 API: 문의 저장 (PRD 01 FR-112, FR-113)
작성	Claude (AI), 2026-10-08, 사용자 검토
이전 · 다음	51 `check_rate_limit()` · 53 `POST /inquiries`

1. 하는 일

홈 화면 Contact 폼에서 보낸 문의를 `inquiries` 표에 저장합니다.

상황	결과
로그인하지 않은 사람	저장합니다. `user_id`는 비어 있고, 상태는 `RECEIVED`, 동의 시각은 지금입니다
로그인한 회원	저장하면서 `user_id`를 연결합니다. 이메일은 **폼에 적은 답장 주소**를 그대로 씁니다
처리방침에 동의하지 않음	422 "Please agree to the privacy policy."와 `details.privacy_agreed: "required"`로 응답합니다. 저장하지도 세지도 않습니다
10분 안에 3번째	429로 응답하고 저장하지 않습니다 (51)
저장 실패 (DB 오류)	오류를 그대로 전달하고, 반복 전송 횟수로 **세지 않습니다**

2. 언제 쓰나

쓰는 곳	
`POST /api/v1/inquiries` (53)	라우터가 로그인한 회원(없으면 `None`)과 IP를 넘깁니다
답변	관리자 화면(SQLAdmin `/admin`, P1-21)에서 `answer`, `answered_at`, 상태를 채웁니다
보관 기간	답변하고 1년이 지나면 지웁니다 (54번 `purge_old_inquiries`)

3. 코드

import (파일 맨 위)

```
import threading
import time
from collections import deque
from datetime import UTC, datetime

from sqlalchemy.orm import Session

from app.core.errors import AppError, ErrorCode
from app.models import Inquiry, User
from app.schemas.inquiry import InquiryCreateRequest
```

`from datetime import UTC, datetime`을 이번에 추가했습니다.

함수

```
def create_inquiry(
    db: Session,
    data: InquiryCreateRequest,
    user: User | None,
    ip: str | None,
) -> Inquiry:
    if not data.privacy_agreed:
        raise AppError(
            ErrorCode.VALIDATION_ERROR,
            422,
            "Please agree to the privacy policy.",
            details={"privacy_agreed": "required"},
        )
    check_rate_limit(ip, data.email)

    inquiry = Inquiry(
        user_id=user.user_id if user else None,
        email=data.email,
        title=data.title,
        content=data.content,
        privacy_agreed_at=datetime.now(UTC).replace(tzinfo=None),
    )
    db.add(inquiry)
    db.commit()
    db.refresh(inquiry)
    record_inquiry_send(ip, data.email)
    return inquiry
```

문법 풀이

식	뜻
`user: User | None`	로그인한 회원이거나 `None`입니다 (P1-03 06번 `OptionalUser`)
`user.user_id if user else None`	조건식입니다. 회원이 있으면 회원 번호를, 없으면 `None`을 씁니다
`datetime.now(UTC).replace(tzinfo=None)`	지금 UTC 시각에서 시간대 표시만 떼어 냅니다 (DB는 항상 UTC로 저장합니다)
`db.refresh(inquiry)`	DB가 채운 값(`inquiry_id`, `created_at`, `status`)을 다시 읽습니다

4. 흐름

1. 처리방침 동의 여부를 확인하고, 동의하지 않았으면 422로 응답합니다.
2. 반복 전송을 검사하고(51), 한도를 넘으면 429로 응답합니다.
3. 문의 행을 만들어 저장합니다.
4. 저장에 성공했으면 반복 전송 1번으로 셉니다.
5. 저장된 문의를 돌려줍니다.

5. 왜 이렇게

선택	이유
동의 검사를 맨 앞에 둠	동의 없이 온 요청은 반복 전송 검사도 하지 않습니다 (횟수로 세지도 않습니다)
세기는 `commit` **뒤에** 함	저장에 실패한 시도는 세지 않는다는 사용자 결정 때문입니다 (51). `commit`이 오류를 던지면 `record_inquiry_send` 줄까지 오지 않습니다 (시험으로 확인했습니다)
로그인한 회원도 **폼에 적은 이메일**로 저장	회원이 다른 주소로 답장을 받고 싶을 수 있기 때문입니다. 회원과의 연결은 `user_id`로 따로 남습니다
`.strip()`을 하지 않음 (템플릿에는 있었음)	50번 스키마가 이미 앞뒤 공백을 지운 값을 넘겨주기 때문입니다
이름(`name`) 칸은 비워 둠	표에는 칸이 있지만 화면 폼에는 이름 칸이 없습니다 (DB는 `NULL`을 허용합니다)
상태는 DB 기본값을 씀	`RECEIVED`입니다. 템플릿과 DB 설계를 그대로 따랐습니다
관리자 알림 메일은 보내지 않음	Phase 1 범위 밖입니다 (템플릿)

6. 시험 (`tests/test_inquiry_service.py`에 5개를 추가해서 모두 17개, 실제 MySQL로 실행)

시험	확인한 것
비회원	DB에 `user_id`가 없고, 이메일, 제목, 내용(줄바꿈 포함), `RECEIVED`, 동의 시각, `created_at`이 저장되며 답변은 비어 있습니다. 반복 전송은 1번으로 셉니다
회원	`user_id`가 연결되고, 이메일은 폼에 적은 다른 주소입니다
동의하지 않음	422와 문구, `details`가 나오고, 저장 0건, 셈 0번입니다
3번째	429이고, 저장은 2건뿐입니다
저장 실패 3번	오류가 그대로 전달되고, 셈은 0번입니다 (사용자 결정)

전체 시험 385개가 통과했고, `ruff` 검사도 통과했습니다. 시험이 끝난 뒤 DB의 문의는 0건입니다.

Backend(FastAPI/Python) 53 — `POST /api/v1/inquiries` (+ `client_ip()`)

항목	내용
파일	`studionx/backend/app/api/v1/inquiries.py`, `app/core/config.py` (설정 하나), `studionx/.env.example` (한 줄)
작업	P1-12 문의 API: 문의 등록 라우터 (PRD 01 FR-112, FR-113)
작성	Claude (AI), 2026-10-08, 사용자 검토
이전 · 다음	52 `create_inquiry()` · 54 `purge_old_inquiries()`

1. 하는 일

홈 화면 Contact 폼에서 보낸 문의를 받습니다. **로그인은 선택 사항이며**, 로그인한 상태라면 문의를 회원과 연결합니다.

요청	응답
`{ email, title: " Bug ", content, privacy_agreed: true }`	**201** `{ "inquiry_id": 12, "created_at": "…" }` 이고, 제목은 `Bug` 로 저장됩니다
로그인 상태 (`Authorization: Bearer …`)	201이고, 회원 번호가 연결됩니다
토큰이 틀림	201이고, 비회원 문의로 받습니다
`privacy_agreed: false`	422이고, `details.privacy_agreed` 에 오류가 담깁니다
이메일 `abc`, 공백만 있는 제목, 내용 없음	422이고, 해당 칸에 오류가 담깁니다
10분 안에 3번째	**429** `TOO_MANY_REQUESTS` "Too many messages. Please try again later."와 `details.retry_after`

2. 언제 쓰나

쓰는 곳	
화면	home.html Contact 폼의 `initContactForm` 입니다 (P1-11에 남은 JS로, 이 작업 뒤에 연결합니다). 201이면 폼을 비우고 안내하며, 429면 문구를 보여 줍니다
이어지는 처리	50번 스키마 → 52번 `create_inquiry` (동의 확인 → 51번 반복 전송 검사 → 저장 → 횟수 세기)

사용자 결정 (2026-10-08): 보낸 사람의 IP를 어디서 읽을 것인가 (결정 A)

상황	읽는 곳
기본값 (`TRUST_PROXY=false`)	연결된 상대의 IP인 `request.client.host` 를 씁니다. **`X-Forwarded-For` 헤더는 무시합니다**
배포할 때 프록시(Nginx 등)를 두고 `TRUST_PROXY=true` 로 설정	`X-Forwarded-For` 의 **첫 번째** IP(원래 보낸 사람)를 씁니다

프록시 뒤에서는 연결 상대가 항상 프록시이기 때문에, 모든 사람이 하나의 IP로 세어집니다. 그래서 헤더가 필요합니다. 하지만 프록시가 **없는데** 헤더를 믿으면, 누구나 헤더를 꾸며서 IP 제한을 피할 수 있습니다. 그래서 기본값은 꺼 두고, 배포할 때 설정 하나로 켜도록 했습니다. (이메일 기준 제한은 어느 쪽이든 동작합니다.)

바꾼 설정 파일	내용
`app/core/config.py`	`trust_proxy: bool = False`와 설명 주석을 추가했습니다
`studionx/.env.example`	`TRUST_PROXY=false`와 설명을 추가했습니다 (실제 `.env`는 건드리지 않았습니다)

3. 코드

import (파일 맨 위)

```
from fastapi import APIRouter, Request, status

from app.api.deps import Db, OptionalUser
from app.core.config import get_settings
from app.schemas.common import ErrorResponse
from app.schemas.inquiry import InquiryCreateRequest, InquiryCreateResponse
from app.services import inquiry_service
```

`get_settings`와 `inquiry_service` 줄을 이번에 추가했습니다.

도우미 함수

```
def client_ip(request: Request) -> str | None:
    if get_settings().trust_proxy:
        first = request.headers.get("x-forwarded-for", "").split(",")[0].strip()
        if first:
            return first
    return request.client.host if request.client else None
```

라우터 (경로와 응답 설정은 템플릿 그대로입니다)

```
@router.post(
    "",
    response_model=InquiryCreateResponse,
    status_code=status.HTTP_201_CREATED,
    responses={422: {"model": ErrorResponse}, 429: {"model": ErrorResponse}},
)
def create_inquiry(
    data: InquiryCreateRequest,
    request: Request,
    user: OptionalUser,
    db: Db,
) -> InquiryCreateResponse:
    inquiry = inquiry_service.create_inquiry(db, data, user, client_ip(request))
    return InquiryCreateResponse(inquiry_id=inquiry.inquiry_id, created_at=inquiry.created_at)
```

문법 풀이

import 문법	뜻
`from app.core.config import get_settings`	`.env` 설정을 읽습니다 (P1-01). 여기서는 `trust_proxy`를 읽습니다
`from app.services import inquiry_service`	모듈 전체를 가져옵니다. 라우터 함수 이름도 `create_inquiry`라서 이름이 겹치지 않게 하기 위해서입니다

식	뜻
`request.headers.get("x-forwarded-for", "")`	헤더 이름은 대소문자를 구분하지 않고, 헤더가 없으면 빈 문자열입니다
`.split(",")[0].strip()`	`"203.0.113.5, 10.0.0.1"`에서 `"203.0.113.5"`만 꺼냅니다 (맨 앞이 원래 보낸 사람이고, 뒤는 거쳐 온 프록시들입니다)
`request.client.host if request.client else None`	연결 상대의 IP입니다. 알 수 없으면 `None`이고, 이때 51번은 이메일로만 셉니다
`user: OptionalUser`	토큰이 있고 올바르면 회원이고, 없거나 틀리면 `None`입니다 (P1-03 06). 401로 응답하지 않습니다

4. 흐름

1. 본문을 검사합니다 (50). 틀리면 422로 응답합니다.
2. 토큰이 있으면 회원을 확인합니다 (없거나 틀리면 비회원으로 처리합니다).
3. 보낸 사람의 IP를 정합니다 (`client_ip`).
4. 52번 `create_inquiry`가 동의 확인(422), 반복 전송 검사(429), 저장을 차례로 합니다.
5. 201과 함께 문의 번호와 접수 시각을 돌려줍니다.

5. 왜 이렇게

선택	이유
로그인은 선택이고, 틀린 토큰도 받아 줌	문의는 누구나 보낼 수 있어야 합니다. 만료된 토큰 때문에 문의가 막히면 안 됩니다
응답에는 번호와 시각만 담음	템플릿을 따랐습니다. 화면은 "보냈습니다"라는 안내만 보여 주므로, 받은 내용을 그대로 돌려줄 필요가 없습니다
헤더에서는 **첫 번째** IP를 씀	프록시들은 자기 앞의 주소를 뒤에 덧붙이기 때문에, 맨 앞이 사용자입니다
설정은 `.env`로 바꿈	개발 컴퓨터(프록시 없음)와 배포 서버(프록시 있음)가 코드를 바꾸지 않고도 다르게 동작할 수 있습니다

6. 시험 (`tests/test_inquiry_routes.py`에 15개를 추가해서 모두 16개, 실제 앱과 MySQL로 실행)

시험	확인한 것
비회원	201, 응답 칸 2개이고, DB에 저장되며 제목의 공백이 지워지고 `user_id`는 비어 있습니다
회원, 틀린 토큰	회원이면 연결되고, 틀린 토큰이면 비회원으로 201입니다
동의하지 않음, 형식 오류 3가지	422이고, `details`에 해당 칸이 담깁니다
3번째	429와 코드, 문구, `retry_after` 1~601초가 나옵니다
기본값에서 헤더를 꾸민 경우	IP마다 다른 `X-Forwarded-For`와 다른 이메일로 보내도 3번째는 429입니다. 헤더를 믿지 않습니다
`TRUST_PROXY`를 켠 경우	헤더의 첫 번째 IP로 사람마다 따로 세서 3명 모두 201이고, 연결 IP는 쓰지 않습니다
`client_ip` 5가지	꺼져 있으면 연결 IP, 켜져 있으면 첫 번째 IP(공백 제거)를 쓰고, 빈 헤더나 헤더가 없으면 연결 IP를 씁니다

전체 시험 400개가 통과했고, `ruff` 검사도 통과했습니다. 시험이 끝난 뒤 DB의 문의는 0건입니다.

Backend(FastAPI/Python) 54 — `purge_old_inquiries()`

항목	내용
파일	`studionx/backend/app/workers/retention.py`
작업	P1-12 문의 API: 오래된 문의 지우기 (개인정보 처리방침 §4)
작성	Claude (AI), 2026-10-08, 사용자 검토
이전 · 다음	53 `POST /inquiries` · 55 `retention.main()`

1. 하는 일

하루에 한 번 실행되는 정리 작업 중 **문의**를 담당하는 부분입니다. 오래된 문의를 DB에서 완전히 지웁니다.

문의 (오늘 기준)	결과
답변한 지 400일	지웁니다
받은 지 500일, 답변한 지 300일	**남깁니다**. 답변한 날부터 1년을 셉니다
답변 없이 받은 지 400일	지웁니다 (사용자 결정)
답변 없이 받은 지 100일	남깁니다
답변한 지 정확히 365일	지웁니다

돌려주는 값은 지운 개수입니다.

2. 언제 쓰나

실행	
55번 `main()`	`python -m app.workers.retention`으로 하루에 한 번 실행합니다 (예약 작업은 배포 때 설정합니다). 49번 `purge_withdrawn_accounts`와 함께 실행됩니다

사용자 결정 (2026-10-08): 답변하지 않은 문의

처리방침에는 "답변 후 1년"만 적혀 있어서, 관리자가 답하지 않은 문의는 영원히 남을 수 있었습니다. 그래서 **받은 뒤 1년**이 지나면 지우기로 했습니다 (추천안 A).

문의	기준 날짜	보관 기간
답변함	답변한 날	1년
답변하지 않음	받은 날	1년

3. 코드

import (파일 맨 위)

```
from datetime import UTC, datetime, timedelta

from sqlalchemy import delete, func, or_, select
```

```
from sqlalchemy.orm import Session

from app.core.config import get_settings
from app.models import Inquiry, User
from app.models.user import UserStatus
from app.services.auth_service import REJOIN_WAIT_DAYS, release_withdrawn
```

`delete`, `func`, `Inquiry`를 이번에 추가했습니다.

상수

```
INQUIRY_KEEP_DAYS = 365  # 답변 후 · (답변 없으면) 받은 뒤 보관 기간 (처리방침 §4, 2026-10-08)
```

함수

```
def purge_old_inquiries(db: Session, now: datetime) -> int:
    if now.tzinfo is not None:
        now = now.astimezone(UTC).replace(tzinfo=None)
    cutoff = now - timedelta(days=INQUIRY_KEEP_DAYS)
    since = func.coalesce(Inquiry.answered_at, Inquiry.created_at)
    count = db.execute(delete(Inquiry).where(since <= cutoff)).rowcount
    db.commit()
    return count
```

문법 풀이

import 문법	뜻
`from sqlalchemy import delete, func`	`DELETE` 문을 만들고, SQL 함수를 부릅니다 (`func.coalesce`는 SQL의 `COALESCE`가 됩니다)

식	뜻
`func.coalesce(Inquiry.answered_at, Inquiry.created_at)`	SQL `COALESCE(answered_at, created_at)` 입니다. 답변 시각이 있으면 그 값을, 없으면(`NULL`) 받은 시각을 씁니다. 두 규칙을 **조건 하나**로 표현합니다
`delete(Inquiry).where(…)`	조건에 맞는 행을 DB에서 한 번에 지웁니다 (하나씩 읽어 오지 않습니다)
`.rowcount`	지운 행의 수입니다

4. 흐름

1. 기준 시각을 지금(UTC)으로부터 365일 전으로 정합니다.
2. "답변 시각, 없으면 받은 시각"이 기준 시각보다 오래된 문의를 한 번에 지웁니다.
3. 저장한 뒤 지운 개수를 돌려줍니다.

5. 왜 이렇게

선택	이유
답변이 없으면 받은 뒤 1년	사용자 결정입니다. 답장용 이메일 같은 개인정보가 끝없이 남지 않게 합니다 (저장 기간 제한 원칙)
답변이 있으면 **답변한 날**이 기준	처리방침을 그대로 따랐습니다. 늦게 답한 문의도 답한 뒤 1년은 남깁니다 (시험: 받은 지 500일, 답변한 지 300일 → 남김)

선택	이유
`COALESCE` 조건 하나로 처리	두 규칙을 따로 지우는 것보다 단순하고, `DELETE` 한 번으로 끝납니다
행을 실제로 지움 (탈퇴 회원처럼 값만 비우지 않음)	다른 표가 문의를 가리키지 않기 때문입니다 (템플릿)
시간대 처리는 49번과 같음	한국 시각으로 날짜가 넘어가도 정확히 1년을 셉니다

6. 처리방침 문구

이 결정에 맞춰 처리방침에 "답변하지 않은 문의는 받은 뒤 1년"이라는 문구를 넣고, 버전을 `2026-10-08`로 올렸습니다 (사용자 결정 A). 자세한 내용은 55 §6에 있습니다.

7. 시험 (`tests/test_retention.py`에 3개를 추가해서 모두 11개, 실제 MySQL로 실행)

시험	확인한 것
문의 5가지	답변한 지 400일, 답변 없이 400일, 답변한 지 정확히 365일인 3개는 지워지고, 답변한 지 300일(받은 지 500일)과 답변 없이 100일인 2개는 남습니다
두 번 실행	처음에는 1개, 두 번째에는 0개를 지웁니다
한국 시각 21:00 (= UTC 12:00)	정확히 1년이 된 문의를 지웁니다

전체 시험 403개가 통과했고, `ruff` 검사도 통과했습니다. 시험이 끝난 뒤 DB의 문의는 0건입니다.

Backend(FastAPI/Python) 55 — `retention.main()` (+ 처리방침 버전 2026-10-08)

항목	내용
파일	`studionx/backend/app/workers/retention.py` / 처리방침: `privacy.html`, `home.html`, `join.js`, `schemas/auth.py`, `docs/Privacy_Policy_Draft.md`
작업	P1-12 문의 API: 보관 기간 정리를 하루 한 번 실행하는 명령 (처리방침 §4)
작성	Claude (AI), 2026-10-08, 사용자 검토
이전 · 다음	54 `purge_old_inquiries()` · 56 프로젝트 요청 스키마 (P1-13)

1. 하는 일

명령 한 줄로 **보관 기간 정리 두 가지**를 차례로 실행하고, 결과를 한 줄로 출력합니다.

```
cd studionx/backend
..\.venv\Scripts\python -m app.workers.retention
# → withdrawn accounts: 0, inquiries: 0
```

차례	정리	함수
①	탈퇴한 지 3일이 지난 계정의 개인 정보를 비웁니다	49번 `purge_withdrawn_accounts` (P1-09)
②	답변한 지 1년, 또는 답변이 없다면 받은 지 1년이 지난 문의를 지웁니다	54번 `purge_old_inquiries`

정리하다가 오류가 나면 예외를 그대로 올려 보냅니다. 그러면 Sentry에 보고되고, 명령은 실패로 끝납니다. 예약 작업이 실패를 알아챌 수 있게 하기 위해서입니다.

2. 언제 쓰나

운영체제의 예약 작업으로 **하루에 한 번** 실행합니다 (Windows 작업 스케줄러나 리눅스 cron). 예약 설정은 **배포할 때** 정합니다 (템플릿 머리말). 지금은 손으로 한 번 실행해서 동작을 확인했습니다. 실제 DB에서 Sentry를 끄고 실행했더니 `withdrawn accounts: 0, inquiries: 0`이 출력되고 종료 코드 0으로 끝났습니다 (지울 것이 없는 상태).

3. 코드

import (파일 맨 위)

```
from datetime import UTC, datetime, timedelta

from sqlalchemy import delete, func, or_, select
from sqlalchemy.orm import Session

from app.core.config import get_settings
from app.core.sentry import init_sentry
from app.db.session import SessionLocal
```

```
from app.models import Inquiry, User
from app.models.user import UserStatus
from app.services.auth_service import REJOIN_WAIT_DAYS, release_withdrawn
```

`init_sentry`와 `SessionLocal` 줄을 이번에 추가했습니다.

함수

```
def main() -> None:
    init_sentry(get_settings())
    now = datetime.now(UTC)
    with SessionLocal() as db:
        accounts = purge_withdrawn_accounts(db, now)
        inquiries = purge_old_inquiries(db, now)
    print(f"withdrawn accounts: {accounts}, inquiries: {inquiries}")
```

파일 맨 아래의 `if __name__ == "__main__": main()`은 템플릿 그대로입니다. `python -m app.workers.retention`으로 실행하면 `main()`이 호출됩니다.

문법 풀이

import 문법	뜻
`from app.core.sentry import init_sentry`	서버(P1-01)와 같은 Sentry 설정입니다. 명령으로 따로 실행될 때에도 오류를 보고하기 위해서 씁니다
`from app.db.session import SessionLocal`	DB 세션을 만드는 공장입니다. 서버 요청 밖에서 직접 세션을 만듭니다

식	뜻
`with SessionLocal() as db:`	세션을 열고, 블록이 끝나면(오류가 나더라도) 닫습니다
`datetime.now(UTC)`	시간대가 붙은 지금 시각입니다. 49번과 54번이 이것을 UTC로 바꿔 비교합니다
`f"…{accounts}…"`	문자열 안에 값을 끼워 넣습니다
`python -m app.workers.retention`	파일을 **모듈**로 실행합니다. `app` 패키지를 찾을 수 있도록 `studionx/backend` 폴더에서 실행합니다

4. 흐름

1. Sentry를 켭니다.
2. 지금 시각을 한 번 잽니다.
3. 세션을 열고 탈퇴 계정 정리와 문의 정리를 차례로 실행합니다 (각각 저장합니다).
4. 결과를 한 줄로 출력합니다. 오류가 나면 그대로 올려 보냅니다.

5. 왜 이렇게

선택	이유
기준 시각을 한 번만 잼	두 정리가 같은 "지금"을 기준으로 삼게 해서, 기록을 읽을 때 헷갈리지 않게 합니다
오류를 삼키지 않음	정리가 조용히 실패하면 처리방침에서 약속한 삭제를 지키지 못하게 됩니다. Sentry 보고와 실패 종료 코드로 알아챌 수 있게 했습니다

선택	이유
Sentry를 먼저 켬	그 뒤에 나는 어떤 오류든 보고되게 하기 위해서입니다
결과를 한 줄로 출력	예약 작업 기록에서 날마다 몇 개를 지웠는지 확인할 수 있습니다

6. 처리방침 버전 올림 (사용자 결정 A, 2026-10-08)

54번의 결정("답변하지 않은 문의는 받은 뒤 1년")을 처리방침에 넣었습니다. 그리고 privacy.html에 적힌 규칙("내용을 바꾸면 버전도 올린다")에 따라 버전을 `2026-10-05`에서 **`2026-10-08`**로 올렸습니다. 아직 공개 전(Draft)이라 다시 동의를 받아야 할 사람은 없고, 앞으로 가입하는 사람은 새 버전으로 기록됩니다.

파일	바꾼 것
`privacy.html` §4	"Messages you send us — 1 year after we reply. **If we never reply, 1 year after you send them.**"와 Version 줄
`home.html` 문의 요약	"One year after we reply — **or one year after you send it, if we never reply** — then we delete it."와 Version 줄
`join.js`, `schemas/auth.py`	`PRIVACY_VERSION = '2026-10-08'`입니다. 가입할 때 화면이 보내고 서버가 확인하는 값이라서, 두 값이 같아야 가입이 됩니다
`docs/Privacy_Policy_Draft.md`	상태, §2.4 보관 표, 버전 줄을 고치고 **버전 기록 표**(10-05 첫 확정판, 10-08 문의 보관)와 PDF를 추가했습니다
`tests/test_auth_service.py`	가입 시험 본문의 `"2026-10-05"`를 상수 `PRIVACY_VERSION`으로 바꿨습니다. 버전이 바뀌어도 시험이 따라가게 하기 위해서입니다

7. 시험 (`tests/test_retention.py`에 2개를 추가해서 모두 13개)

시험	확인한 것
`main()` (세션과 Sentry를 시험용으로 바꿔 끼움)	Sentry를 먼저 켜고, 탈퇴한 지 10일 된 계정과 400일 된 문의를 정리하며, `withdrawn accounts: N, inquiries: M`을 출력합니다
정리 중 오류	예외가 그대로 올라옵니다
(손으로) 실제 명령 실행	`withdrawn accounts: 0, inquiries: 0`이 출력되고, 종료 코드는 0입니다

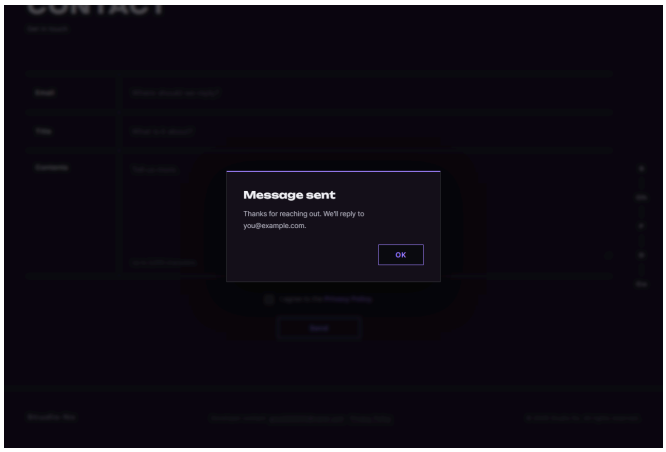
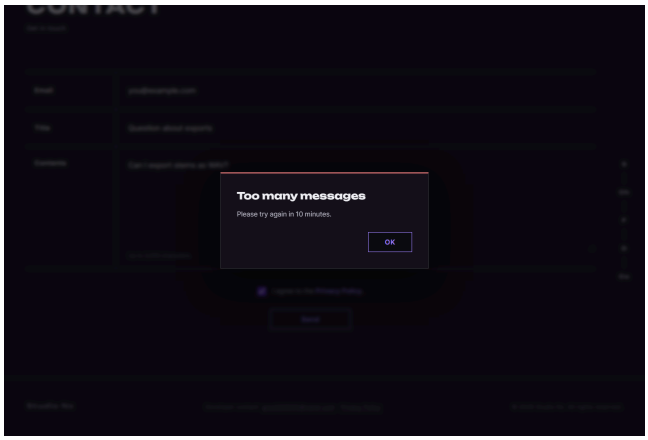
전체 시험 405개가 통과했고(처리방침 버전을 올린 뒤에도 통과), `ruff` 검사도 통과했습니다.

JS 28 — `initContactForm()`

항목	내용
파일	`studionx/frontend/static/js/pages/home.js`
작업	P1-12 문의: 홈 Contact 폼 보내기, 로그인하면 이메일 자동 채움 (PRD 01 FR-112, FR-113; P1-11 화면에서 남은 JS)
작성	Claude (AI), 2026-10-08, 사용자 검토
이전 · 다음	27 `initLogoutButton()` · 29 `formatRelativeTime()` (P1-14)

1. 하는 일

홈 화면 **Contact** 폼의 **Send** 버튼을 서버(백엔드 53)에 연결하고, 결과를 알림창으로 알려 줍니다.

보냄	이미 3번째 (10분에 2번을 넘음)
	

상황	화면
로그인한 사람이 홈을 열면	이메일 칸에 **내 이메일이 미리 채워집니다** (비어 있을 때만)
빈 칸, 형식 틀림, 동의하지 않음	보내지 않고 알림창을 띄웁니다 (FIX-01 19: 브라우저 검사)
보내는 중	Send가 잠깁니다
성공	알림 "Message sent — Thanks for reaching out. We'll reply to you@example.com."이 뜨고, 폼을 비웁니다 (내 이메일은 다시 채웁니다)
10분에 3번째 (서버 429)	빨간 알림 "Too many messages — Please try again in 10 minutes."가 뜨고, 폼은 그대로 둡니다
그 밖의 서버 오류	빨간 알림 "Couldn't send your message"와 서버 문구가 뜨고, 폼은 그대로 둡니다

2. 언제 쓰나

home.html이 열릴 때 한 번 부릅니다. 이어지는 것들은 다음과 같습니다.

순서	하는 곳
① 칸 검사와 알림	브라우저의 기본 검사와 FIX-01 19 `initFormAlerts` 입니다. 통과해야 `submit` 이 일어납니다
② 보내기	백엔드 53 `POST /inquiries` → 50, 51, 52

순서	하는 곳
③ 결과	FIX-01 18 `showAlert`
(자동 채움)	P1-02 `isLoggedIn` → P1-08 `GET /users/me`

결과를 알림창으로 보여 주므로 **HTML은 바꾸지 않았습니다** (폼에 결과 문구 자리가 없었습니다).

3. 코드

import (`home.js` 맨 위)

```
import { initAuthState, initLoginForm, initPasswordToggles, initOAuthPlaceholders } from '../core/auth-u
i.js';
import { initModalFallback } from '../core/modal.js';
import { includePartials } from '../core/include.js';
import { parseLoginHash, isLoggedIn } from '../core/auth.js';
import { initFormAlerts, showAlert } from '../core/dialog.js';
import { api } from '../core/api.js';
```

`isLoggedIn`, `showAlert`, `api`를 이번에 추가했습니다.

함수

```
export function initContactForm() {
  const form = document.getElementById('contact-form');
  const email = document.getElementById('contact-email');
  const title = document.getElementById('contact-subject');
  const content = document.getElementById('contact-content');
  const agree = document.getElementById('contact-agree');
  if (!form || !email || !title || !content || !agree) return;
  const submit = form.querySelector('button[type="submit"]');
  let myEmail = '';

  // 로그인 상태면 내 이메일을 미리 채움 (기다리지 않음 — 폼은 바로 쓸 수 있음)
  isLoggedIn().then(async (loggedIn) => {
    if (!loggedIn) return;
    try {
      myEmail = (await api.get('/users/me')).email;
    } catch {
      return; // 못 받아도 문의는 직접 적어 보낼 수 있음
    }
    if (email.value === '') email.value = myEmail;
  });

  form.addEventListener('submit', async (event) => {
    event.preventDefault();
    if (submit) submit.disabled = true;
    try {
      const sentTo = email.value.trim();
      // 토큰을 붙여 보냄 → 로그인한 회원이면 서버가 문의를 회원과 연결 (FR-112).
      // 토큰이 없거나 틀려도 이 API 는 401 이 아니라 비회원으로 받음 (백엔드 53)
      await api.post('/inquiries', {
        email: sentTo,
        title: title.value,
```

```
        content: content.value,
        privacy_agreed: agree.checked,
      });
      form.reset();
      if (myEmail) email.value = myEmail;
      showAlert('Message sent', `Thanks for reaching out. We'll reply to ${sentTo}.`); // 닫힐 때까지 기다리지 않음 — 버튼은 바로 다시 켬
    } catch (err) {
      if (err.status === 429) {
        const wait = err.details && err.details.retry_after ? err.details.retry_after : 600;
        const minutes = Math.ceil(wait / 60);
        showAlert(
          'Too many messages',
          `Please try again in ${minutes} minute${minutes === 1 ? '' : 's'}.`,
          { tone: 'error' },
        );
      } else {
        showAlert("Couldn't send your message", err.message, { tone: 'error' });
      }
    } finally {
      if (submit) submit.disabled = false;
    }
  });
}
```

최신 문법으로 쓰면 (공부용이며, 바뀌는 부분만 적었습니다)

```
const wait = err.details?.retry_after ?? 600;
…
const { email: mine } = await api.get('/users/me');
email.value ||= mine;
```

최신 문법	실제 코드	뜻
`err.details?.retry_after ?? 600`	`err.details && err.details.retry_after ? err.details.retry_after : 600`	있으면 그 값을, 없으면 600을 씁니다
`const { email: mine } = …`	`myEmail = (await …).email`	응답에서 칸을 꺼내면서 이름을 바꿉니다
`a ||= b`	`if (a === '') a = b`	비어 있을 때만 넣습니다

문법 풀이

import 문법	뜻
`import { …, isLoggedIn } from '../core/auth.js'`	P1-02 06번입니다. 토큰이 없으면 재발급을 시도해서 로그인 여부를 알려줍니다
`import { …, showAlert } from '../core/dialog.js'`	FIX-01 18번의 알림창입니다

식	뜻
`isLoggedIn().then(async (loggedIn) => { … })`	**기다리지 않고**, 결과가 오면 실행합니다. 그동안에도 폼을 쓸 수 있습니다
`catch { … }` (괄호 없음)	오류 내용을 쓰지 않을 때 이렇게 줄여 씁니다 (`api.js`도 같은 방식입니다)

식	뜻
`form.reset()`	모든 칸을 처음 값으로 되돌립니다 (체크박스도 해제됩니다)
`Math.ceil(wait / 60)`	초를 분으로 바꾸며 올림합니다 (예: 599초 → 10분)
`` `minute${minutes === 1 ? '' : 's'}` ``	1이면 minute, 아니면 minutes가 됩니다
`showAlert(…)` (`await` 없음)	알림창이 닫히기를 **기다리지 않습니다** (아래 §5 참고)

4. 흐름

페이지를 열 때: 로그인 상태를 확인하고, 로그인 상태면 내 이메일을 받아서 칸이 비어 있을 때 채웁니다. **보낼 때:** (브라우저 검사를 통과하면) Send를 잠그고 보냅니다. 성공하면 폼을 비우고(내 이메일은 다시 채움) 알림을 띄우고, 429면 "몇 분 뒤"를, 그 밖의 오류면 서버 문구를 보여 준 뒤 Send를 다시 켭니다.

5. 왜 이렇게

선택	이유
결과를 알림창으로 보여 줌	폼에 결과 문구 자리가 없고, FIX-01에서 모든 알림을 같은 모양으로 쓰기로 했습니다
알림창을 **기다리지 않음**	시험에서 찾은 문제입니다. 처음에는 `await showAlert(…)`였는데, 그러면 사용자가 OK를 누를 때까지 `finally`(Send 다시 켜기)가 실행되지 않았습니다
막히거나 실패하면 폼을 비우지 않음	적어 둔 문의가 사라지면 다시 써야 합니다
429는 "몇 분 뒤"로 보여 줌	서버가 `retry_after`(초)를 줍니다(51). 사람은 분 단위로 읽는 편이 편합니다
토큰을 붙여 보냄 (`api.post`의 기본값)	로그인한 회원이면 서버(52)가 문의에 `user_id`를 연결합니다 (FR-112). 이 API는 토큰이 없거나 틀려도 401이 아니라 비회원으로 받으므로(53), 문의가 막히지 않습니다
자동 채움은 칸이 **비어 있을 때만**	사용자가 먼저 다른 주소를 적었다면 덮어쓰지 않습니다
자동 채움을 기다리지 않음	로그인 확인과 내 정보 요청이 진행되는 동안에도 폼을 바로 쓸 수 있게 합니다

처음 만든 것과 바꾼 점 (회원 연결): 처음에는 `{ auth: false }`(토큰 없이)로 보냈는데, 그러면 로그인한 회원의 문의도 **회원과 연결되지 않았습니다** (FR-112의 빈틈). 이 API는 토큰 문제로 401을 내지 않으므로 토큰을 붙여 보내도록 고쳤습니다. DB에서 회원의 문의는 `user_id`가 연결되고, 비회원의 문의는 비어 있음을 확인했습니다.

6. 시험 (실제 home.html, 실제 서버, 9개)

반복 전송 제한은 서버 메모리에서 세므로, 시험마다 시험 서버를 다시 켜서 기록을 비웠습니다.

시험	확인한 것
비회원	이메일 칸이 비어 있습니다
보내기	보내는 동안 Send가 잠기고, "Message sent"와 받는 주소가 나오며, 폼이 비워지고 Send가 다시 켜집니다
3번째	빨간 "Too many messages"와 "Please try again in 10 minutes."가 나오고, 폼은 그대로입니다
서버 오류 (가짜 500)	"Couldn't send your message"와 서버 문구가 나옵니다

시험	확인한 것
로그인한 사람	이메일이 미리 채워지고, 보낸 뒤 폼을 비워도 내 이메일은 다시 채워집니다. **DB에서 그 문의의 `user_id`는 그 회원입니다** (비회원 문의는 비어 있습니다)

처음에는 6/7이었습니다. §5의 "알림을 기다려서 Send가 잠긴 채로 남는 문제"를 찾아 고친 뒤 7/7, 로그인 시험 2/2가 되었습니다. 회원 연결 빈틈(위)도 고친 뒤 다시 돌려서 통과했습니다. 홈 화면의 앞선 시험(P1-07 13, 14, FIX-01 18, 19)도 다시 돌려서 모두 통과했습니다. 시험이 만든 문의와 계정은 지웠습니다.